Visual SLAM

dihuang

Preview

- Introduction
- Typical Pipeline
- Front-end: Visual Odometry
- Back-end: Optimization
- PTAM
- ORB-SLAM vs. LSD-SLAM vs. SVO-SLAM

What is SLAM?

- Simultaneous Localization And Mapping
- SLAM = Localization (locating the sensor) + Mapping (building a map)

- A chicken-and-egg problem
  - A map is needed for localization
  - A pose estimate is needed for mapping

- Formulation
  - Motion Function $x_k = f(x_{k-1}, u_k, w_k)$
  - Observation Function $z_{k,j} = h(y_j, x_k, v_{k,j})$

ORB-SLAM2 DEMO

- https://www.youtube.com/watch?v=ufvPS5wJAx0

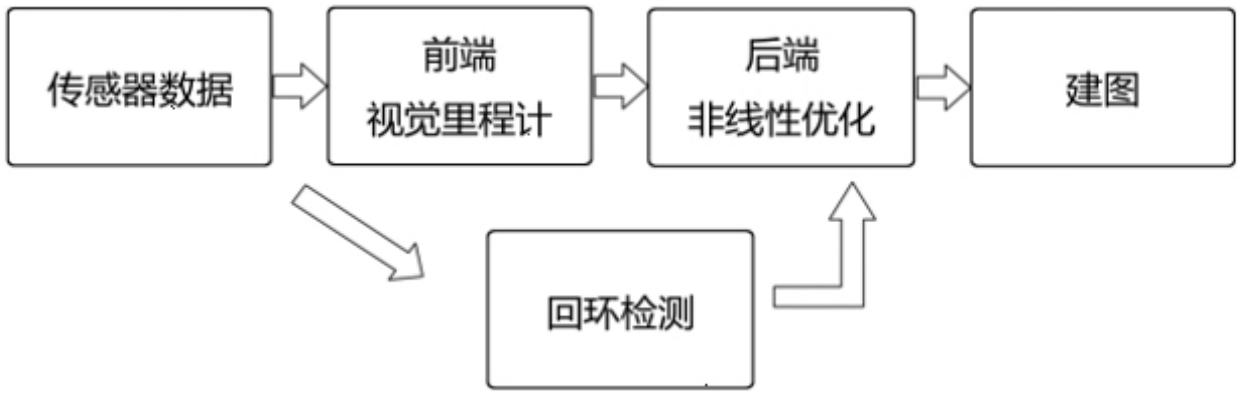
Why we need SLAM?

- Directly lead to the study of sensor fusion under more challenging setups.
- Understand the true topology of the environment.
- Needed for many applications that, either implicitly or explicitly, do require a globally consistent map.
  - AR Applications: https://www.youtube.com/watch?v=bqDZW9y6RCo
  - Robots Applications: https://www.youtube.com/watch?v=bq5HZzGF3vQ

-- Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age

Typical SLAM Pipeline

- Sensor.
- Front-end: Visual Odometry.
- Back-end: Optimization.
- Loop Closure.
  - Bag-of-Words, BoW.
- Mapping.
  - Occupancy Grid Map
  - Point Cloud
  - Mesh



Front-end: Visual Odometry

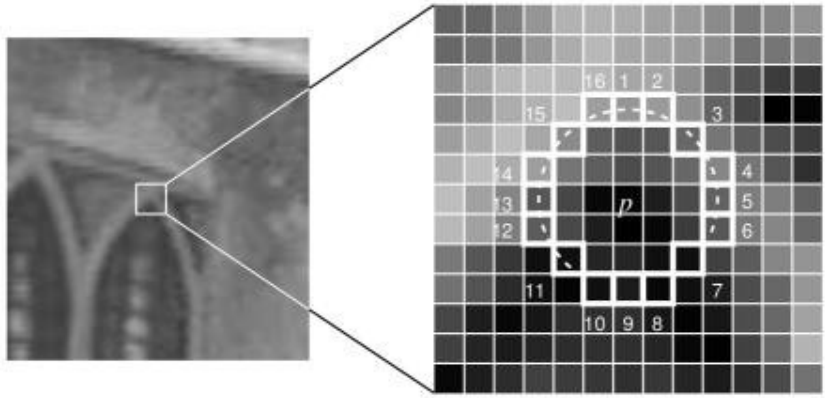
- Feature vs. Direct.

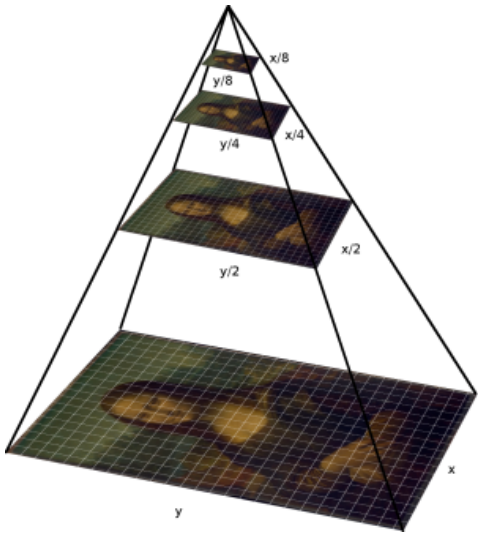
Front-end: Visual Odometry

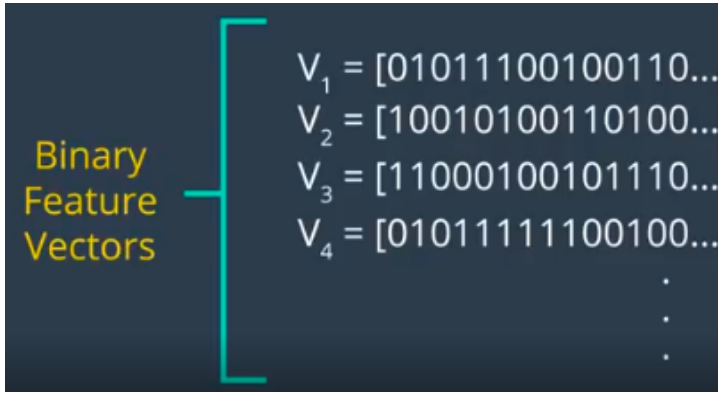
- Feature vs. Direct.
- Feature Extraction.
  - SIFT vs. SURF vs. ORB …
  - 1000 feature points, SIFT : SURF : ORB = 5228.7ms : 217.3ms : 15.3ms

ORB

- Oriented FAST + Rotated BRIEF
- https://medium.com/@deepanshut041/introduction-to-orb-oriented-fast-and-rotated-brief-4220e8ec40cf







Front-end: Visual Odometry

- Feature vs. Direct.
- Feature Extraction.
  - SIFT vs. SURF vs. ORB …
  - 1000 feature points, SIFT : SURF : ORB = 5228.7ms : 217.3ms : 15.3ms
- Epipolar Constraint.
  - Essential Matrix
  - Fundamental Matrix
  - Homography Matrix

Epipolar Constraint

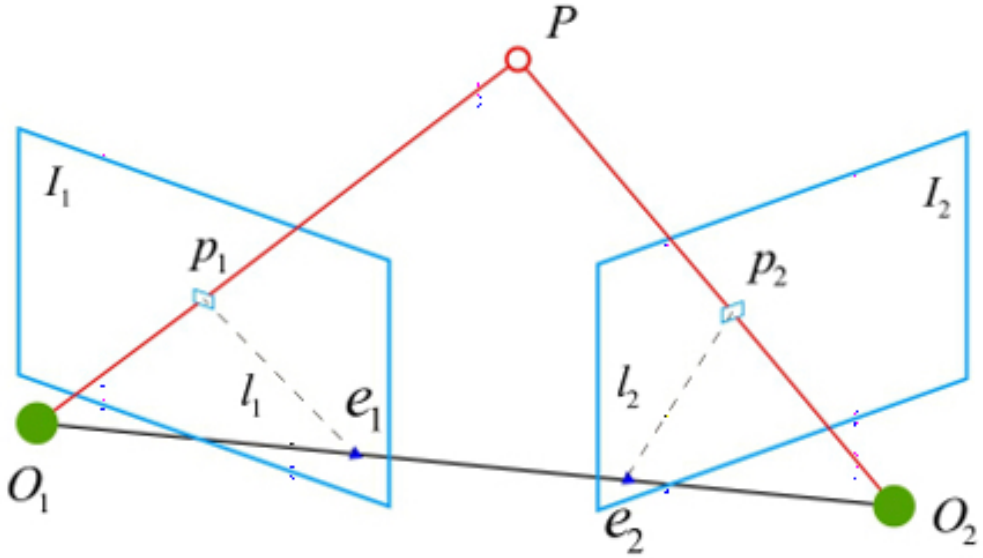
$p_1 = KP, \qquad p_2 = K(RP + t)$

$x_1 = K^{-1}p_1, \qquad x_2 = K^{-1}p_2$

$x_2 = Rx_1 + t$

$t \times x_2 = t \times Rx_1$

$x_2^T t \times x_2 = x_2^T t \times Rx_1$



$x_2^T t \times Rx_1 = 0$

$p_2^T K^{-T} t \times RK^{-1} p_1 = 0$

Epipolar Constraint

- Essential Matrix: $E = t \times R$
- Fundamental Matrix: $F = K^{-T} E K^{-1}$
- Epipolar Constraint: $x_2^T E x_1 = p_2^T F p_1 = 0$

- Camera pose estimation:
  - Step1, recover the essential matrix E from the epipolar constraints associated with a set of point pairs.
  - Step2, Extract the relative translation and rotation from the essential matrix E.

Eight-point-algorithm (Longuet-Higgins ’81)

- Compute an approximation of the essential matrix.

$$\begin{pmatrix} u_1 & v_1 & 1 \end{pmatrix} \begin{pmatrix} e_1 & e_2 & e_3 \\ e_4 & e_5 & e_6 \\ e_7 & e_8 & e_9 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ 1 \end{pmatrix} = 0$$

$$e = \begin{bmatrix} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 & e_9 \end{bmatrix}^T$$

$$E = U \Sigma V^T$$

- Project onto essential space.

$$E = U \mathrm{diag}\{\sigma_1, \sigma_2, \sigma_3\} V^T$$

normalized essential space: $\sigma_1, \sigma_2, \sigma_3$ ------ 1,1,0

- Recover the displacement from the essential matrix.

Eight-point-algorithm (Longuet-Higgins ’81)

- Compute an approximation of the essential matrix.
- Project onto essential space.
- Recover the displacement from the essential matrix.
  - 4 possible solutions.

$$R = UR_Z^\top(\pm\tfrac{\pi}{2})V^\top, \quad \widehat{T} = UR_Z(\pm\tfrac{\pi}{2})\Sigma U^\top,$$

with a rotation by $\pm\frac{\pi}{2}$ around z:

$$R_Z^\top(\pm\tfrac{\pi}{2}) = \begin{pmatrix} 0 & \pm 1 & 0 \\ \mp 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Planar Homographies

- Homography Matrix Definition

$$n^T P + d = 0 \rightarrow -\frac{n^T P}{d} = 1$$

$$p_2 = K(RP + t) = K\left(R - \frac{tn^T}{d}\right)K^{-1}p_1 \rightarrow p_2 = Hp_1$$

- Planar Homography Constraint
- Solution

$$\begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = \begin{pmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix}$$

Front-end: Visual Odometry

- Feature vs. Direct.
- Feature Extraction.
  - SIFT vs. SURF vs. ORB …
  - 1000 feature points, SIFT : SURF : ORB = 5228.7ms : 217.3ms : 15.3ms
- Epipolar Constraint.
  - Essential Matrix
  - Fundamental Matrix
  - Homography Matrix
- Triangulation

$$s_1 x_1 = s_2 R x_2 + t$$
$$s_1 x_1 \times x_1 = 0 = s_2 x_1 \times R x_2 + x_1 \times t$$

Back-end: Optimization

- Formulation
  - Motion Function $x_k = f(x_{k-1}, u_k, w_k)$
  - Observation Function $z_{k,j} = h(y_j, x_k, v_{k,j})$

Back-end: Optimization

- Formulation
  - Motion Function $x_k = f(x_{k-1}, u_k, w_k)$
  - Observation Function $z_{k,j} = h(y_j, x_k, v_{k,j})$

- Extended Kalman Filter, EKF.
  - Pros: Easy, Low computation cost.
  - Cons: Limitation of First-order Markov, Limitation of First-order Taylor expansion, high storage request.
- Bundle Adjustment, BA.
- Pose Graph.

Bundle Adjustment

- Cost Function

$$\frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n}||e_{ij}||^2 = \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n}||z_{ij} - h(\xi_i, p_j)||^2$$

- Sparse Structure
- Robust Kernel

Visual SLAM by SFM

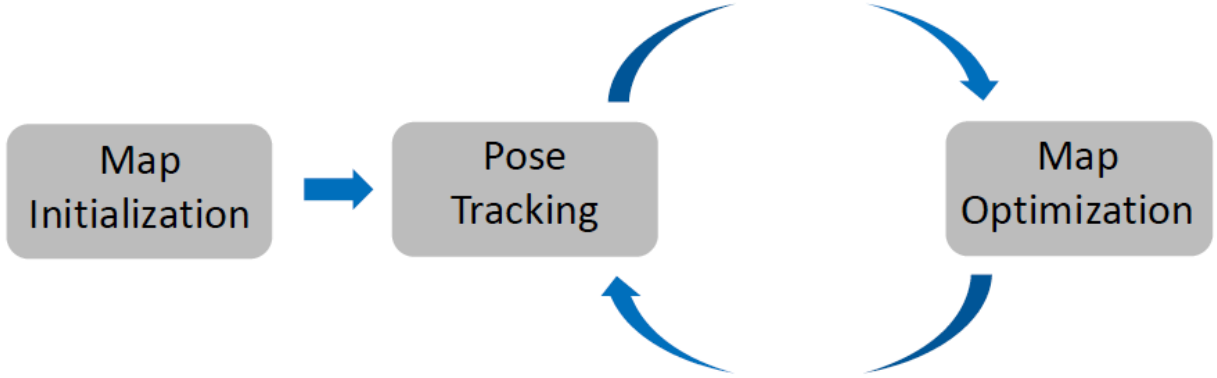
- Solve an incremental SfM at every new frame

- Realtime constraint (many trade-offs for better efficiency)
  - Keyframe based mapping (only a subset of frames are used for mapping)
  - Local BA (bundle adjustment with only nearby video frames)

- Sequential video input
  - Sorted input images (match each image to its previous frame)
  - Regular time interval between frames (motion model to facilitate matching)

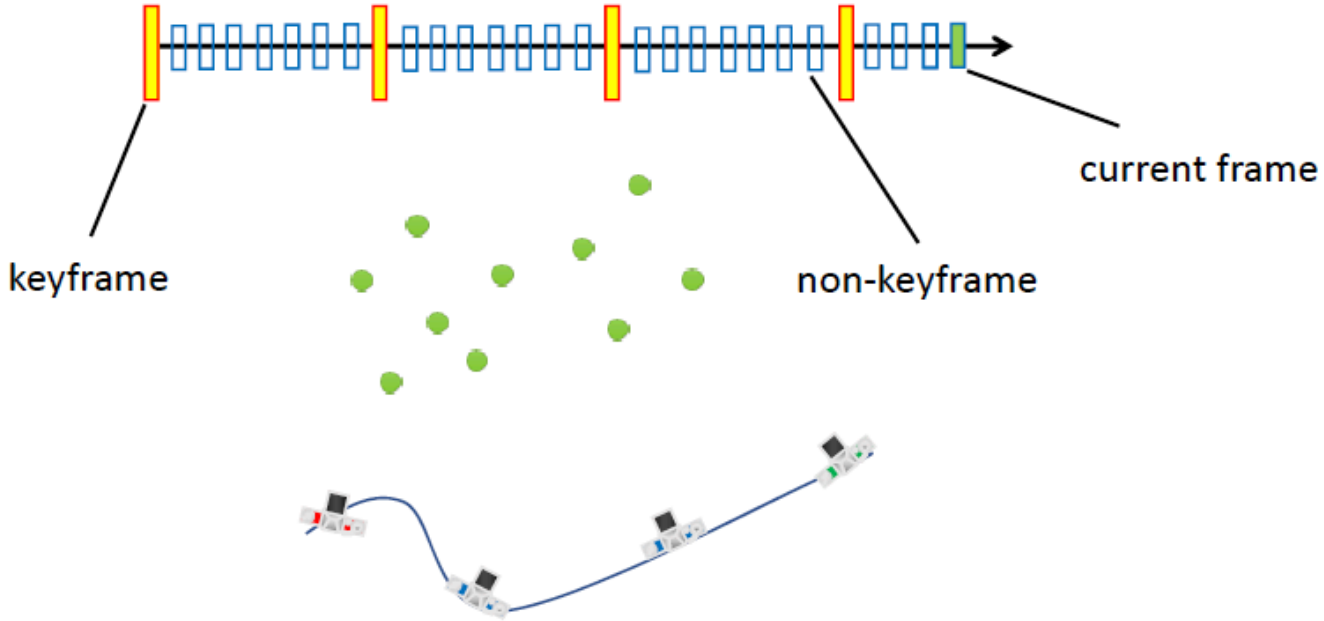
SLAM Pipeline

- Initialization: (essential matrix, triangulation)
- Pose Tracking:
  - Feature tracking
  - Pose-only BA
- Map Optimization:
  - Triangulation, BA
  - Loop closure, pose-graph (today)



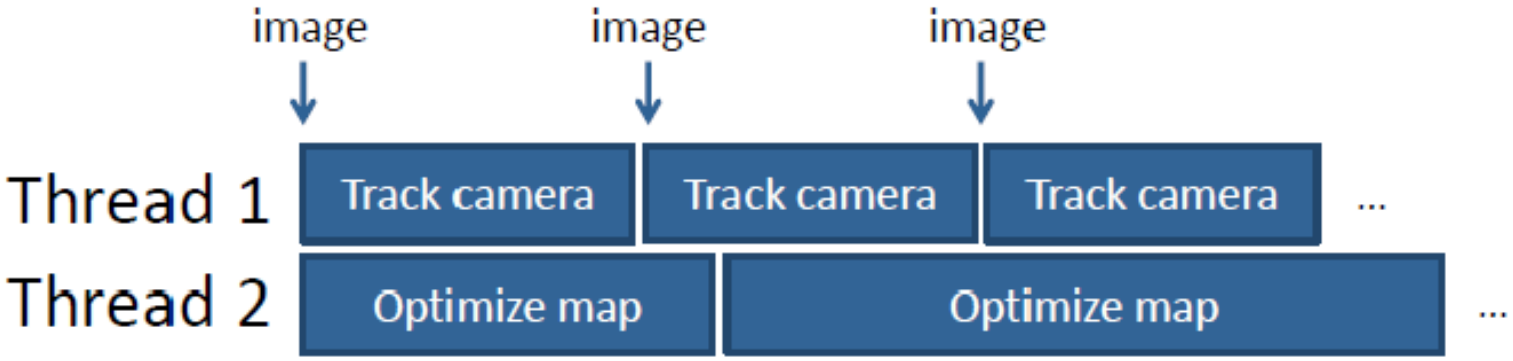
Key-frame based Visual SLAM

- Solving camera pose for every input frame
- Only use some “keyframes” to triangulate/optimize map points



Parallel Tracking and Mapping [ISMAR 2007]

- Parallel tracking and mapping
  - A real time tracking thread runs in real-time (30Hz)
  - An offline mapping thread for map maintenance



Tracking

- Step 1: feature correspondence
  - Generate 8x8 matching template (warped from keyframe)
  - Search for correspondence in a fixed radius around projected position
    - Only search at pre-detected corner points (e.g. FAST points)

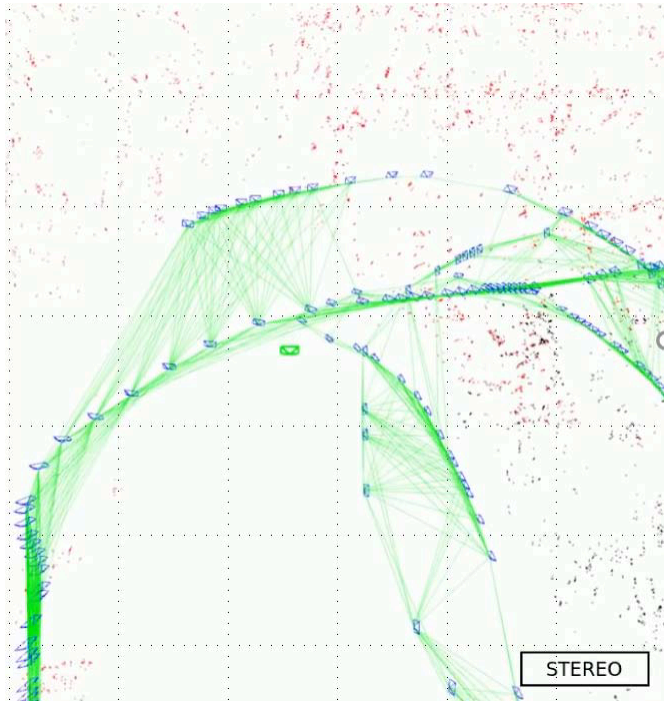
- Step 2: solve camera motion
  - With camera pose initialized by extrapolation, e.g. constant velocity motion
  - Perform local camera only BA (BA with map points fixed)
  - camera might also be initialized by PnP

Mapping

- Triggered by the insertion of a new keyframe
- Triangulate any tracked image corners that are not reconstructed
- Run corner detection (e.g. Harris) to generate more points for tracking
- Call local BA to optimize both points and poses
  - BA is slow, may be called after several keyframe insertions

Keyframes

- Defined heuristically, various from system to system
- A keyframe is typically inserted when:
  - Camera tracking is robust
  - There are insufficient corner points to track
  - There is a large camera motion (rotation or translation)
  - There is no keyframe inserted for a while (e.g. more than 30 frames)
  - …



Re-localization

- Tracking can lose due to various reasons
  - Motion blurs
  - Moving objects
  - Large occlusion
  - Sudden fast motion
  - Sudden illumination change

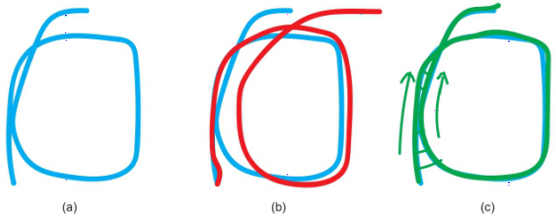
- Re-localization is to recover from such a sudden tracking failure

Re-localization

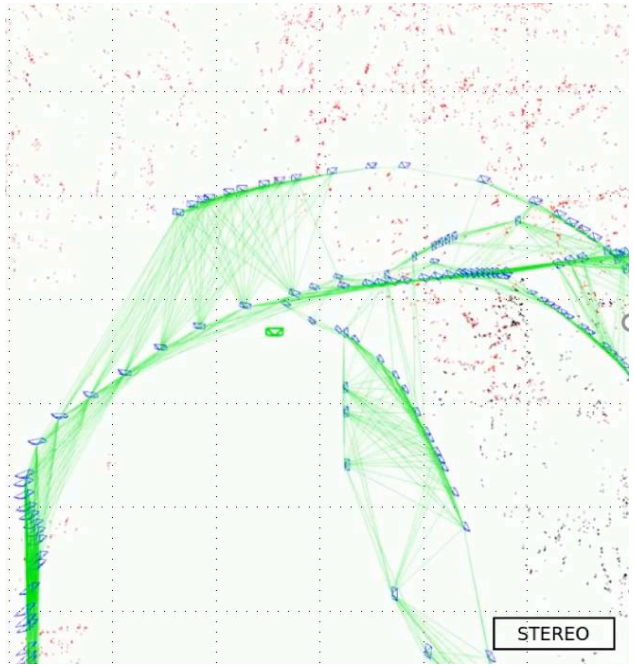
- Re-localization typically includes the following steps
  - Image search: search the current frame among the pre-indexed keyframes
    - E.g. by bag-of-words models
    - It returns a keyframe with sufficient view overlap with the current frame
  - Feature matching between these two frames
  - PnP and local BA to register the current frame to the map
  - Continue the original SLAM

Loop Closure

- Loop detection
  - Identify if a loop exist
  - Again, by the same image search technical in re-localization
  - Typically, applied to every newly inserted keyframe



- Construct a pose-graph for the next step
  - Where a keyframe is a vertex
  - Two keyframes are connected if they have view overlap



Loop Closure

- Loop optimization
  - In the simplest case, a direct BA can generate good result
  - In many cases, the drifting error is too large to be corrected by BA

- Most of the time, a global SfM is desirable
  - Solving all keyframe poses from input pairwise relative motion constraints
  - Update map points afterwards
  - Referred as ‘pose-graph optimization’ in robotics
    - the keyframe graph is a graph with camera poses (no 3D points)
  - The most challenging problem is to deal with wrong loops (due to repetitive structures)

Thanks